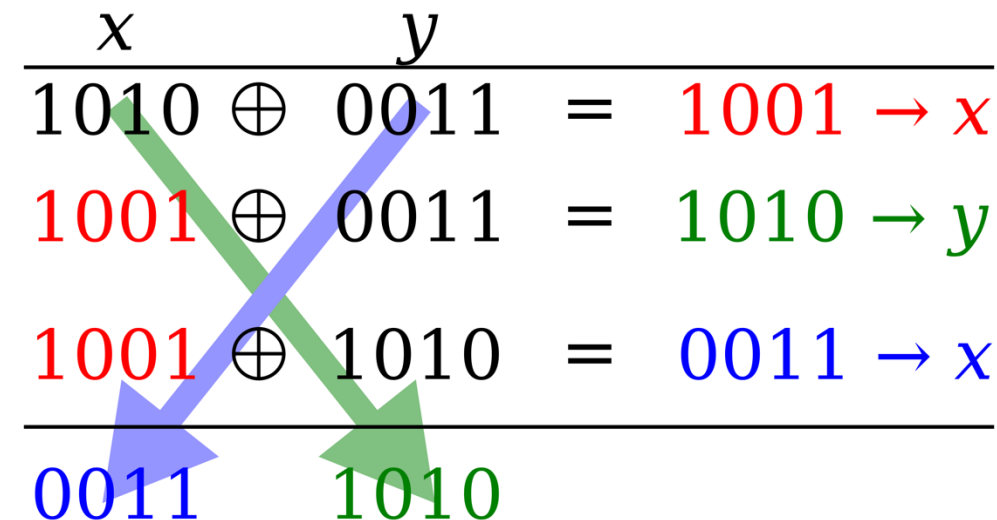


Code of the Day

- What does the following function do?

```
void Function(int& x, int& y) {  
    x ^= y;  
    y ^= x;  
    x ^= y;  
}
```



$X \oplus X = 0$ (XOR-ing a value with itself results in 0)

$X \oplus 0 = X$ (XOR-ing a value with 0 results in the same value)

$X \oplus Y \oplus Y = X$ (XOR-ing a value with another value twice restores the original value)

ECE 759

High Performance Computing for Applications in Engineering

[Spring 2025]

From Code to Machine Instructions

2025/01/27

Before we get started...

- **Purpose of today's lecture:**
 - Discuss the basics of how the hardware works for the purpose of writing better software
 - Discuss how a sequence of instructions process a bunch of data
- **Miscellaneous**
 - Homework went out already. Due on Feb 3 12:59 PM via Canvas

Assignment, the typical work cycle

- 1) Write your solution using your favorite editor/IDE on **your** computer or CAE workstation
- 2) Build & test the code on **your/CAE** computer until you are convinced that it works as intended
- 3) Get your files over to Euler using `scp` or via `git`
- 4) Build your executable on Euler, on the log-in node
- 5) Write/Recycle `slurm` scripts to define how your executable will be managed by Slurm
- 6) Run `slurm` script to place your executable in the Slurm queue
- 7) Once it finished, examine the output of your job
 - By default, output of your program is written to a file starting with "slurm-"
- 8) If there are any small problems, use a terminal text editor on Euler (like `vim`) to make minor changes to your code and then rerun your `slurm` script to test your program again on Euler
NOTE: If the results look bad, you might need to go back to 1)
- 9) Once you are convinced that program works as expected on Euler, push solution into your `git` repo
 - Note: Your code will be graded based on how it runs **on Euler**, not how it works on your computer

From C/C++ Code to Machine Instructions (1/4)

- Your program is the result of a compilation step in which a source file is turned into an executable
 - Source code is for programmers – human readable
 - Machine code is for machine – machine readable and (usually) not human readable
- What's an “**executable**”?
 - Sequence of machine instructions that can be processed, one by one, by the processor
 - What are instructions? An embedded binary sequence to instruct the process to perform a specific job
 - EX: load data, compute data, store result
- Execution finishes when your program is done at which the last instruction completes

```
#include <iostream>
/* Simplest C++ Program */
int main(int argc, char **argv) {
    std::cout << "Hello World!\n";
    return 0;
}
```



```
~$ gcc -o helloWorld helloWorld.cpp
~$ ls
helloWorld  helloWorld.cpp
~$ ./helloWorld
Hello World!
```

From C Code to Machine Instructions (2/4)

- There is a difference between a line of code and a machine instruction

- Example:

- Line of C code: `a[4] = delta + a[3];` *//line of C code*

- MIPS *assembly code* generated by the compiler (three instructions):

```
lw  $t0, 12($s2) # reg $t0 gets value stored 12 bytes from address in $s2
add $t0, $s4, $t0 # reg $t0 gets the sum of values stored in reg $s4 and reg $t0
sw  $t0, 16($s2) # store what's in $t0 at mem location 16 bytes from address in $s2
```

- Set of three corresponding *MIPS instructions* produced by the compiler:

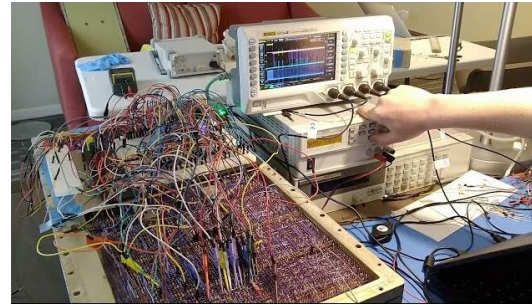
```
10001110010010000000000000001100
00000010100010000100000000100000
10101110010010000000000000001000
```

From C Code to Machine Instructions (3/4)

- **C code** – what you write to implement an algorithm (human readable)
- **Intermediate Representation** – compiler converts your code to a specialized representation that it can optimize
 - Low-level virtual machine (LLVM) is one such example to represent C/C++ in IR
- **Assembly code** – another intermediate step for compiler, humans can still read it
 - Go check compiler explorer: <https://godbolt.org>
- **Machine code/Instructions** – what the assembly code gets translated into what the control unit (CU) digests
 - Machine code: what you see in an editor (`notepad`, `vim`, etc.) if you open an executable file
 - The file content is not human readable
 - There is a **direct** correspondence between a line of assembly code and instructions
 - This is often one-to-one; particularly true for certain processors)

From C Code to Machine Instructions (4/4)

- The compiler goes from C code to machine instructions without producing anything else (unless you ask for it)
 - That is, no assembly is saved for you to peek at (again, unless you ask the compiler for it)
- Back in the day, people programmed computers via assembly code
 - Original Apollo 11 Guidance Computer (AGC) for the command and lunar modules are written in assembly
 - Program is open-source at <https://github.com/chrislgarry/Apollo-11>
 - Ya... quite amazing...



- Today writing assembly code done only for critical parts of a program by people who want to highly optimize the execution and choose to overrule the compiler – Yet, in my experience, today's compilers can do a very good job in optimizing your code to assembly; you don't really need to do this on your own

More on the “assembly instructions” side of things

- We'll not cover assembly in this course
- If you really are into computing though, you should be able to handle assembly code
 - In critical cases you might want to check that the compiler does what you think it should do
 - Go check compiler explorer: <https://godbolt.org>
 - Examples: you optimize the code with `-O3` and want to confirm that
 - A for-loop got unrolled to reduce the number of jump conditions and improve changes for pipelining
 - A function got inlined to avoid a pair of jumps that ruin pipelining

Same code, same compiler (gcc), different flags

```
1 // ME759 example, from godbolt.org
2 int square(int num) {
3     return num * num;
4 }
```

No gcc flag used here



```
1 square(int):
2     push    rbp
3     mov     rbp, rsp
4     mov     DWORD PTR [rbp-4], edi
5     mov     eax, DWORD PTR [rbp-4]
6     imul    eax, eax
7     pop     rbp
8     ret
```



-O3 flag (optimization) used here

```
1 square(int):
2     mov     eax, edi
3     imul    eax, edi
4     ret
```

eax: 32-bit general-purpose register whose main purpose in life (but not the only one) is to store the return value of a function

Example: the same C code
leads to different assembly code on different
processor architectures (x86 and MIPS)

```
int main(){
    const double fctr = 3.14/180.0;
    double a = 60.0;
    double b = 120.0;
    double c;
    c = fctr*(a + b);
    return 0;
}
```

C code

MIPS ISA

x86 ISA

Intermediate
Representation
(e.g., LLVM) +
Optimization

Legend:
fldl load floating point
fmlulp multiply & pop register stack
movl move from one register to another

```
call    __main
        fldl    LC0
        fstpl   -40(%ebp)
        fldl    LC1
        fstpl   -32(%ebp)
        fldl    LC2
        fstpl   -24(%ebp)
        fldl    -32(%ebp)
        faddl   -24(%ebp)
        fldl    LC0
        fmlulp  %st,
%st(1)  %st,
        fstpl   -16(%ebp)
        movl    $0, %eax
        addl    $36, %esp
        popl    %ecx
        popl    %ebp
        leal    -4(%ecx),
%esp
        ret
LC0:
        .long   387883269
        .long   1066524452
        .align  8
LC1:
        .long   0
        .long   1078853632
        .align  8
LC2:
        .long   0
        .long   1079902208
```

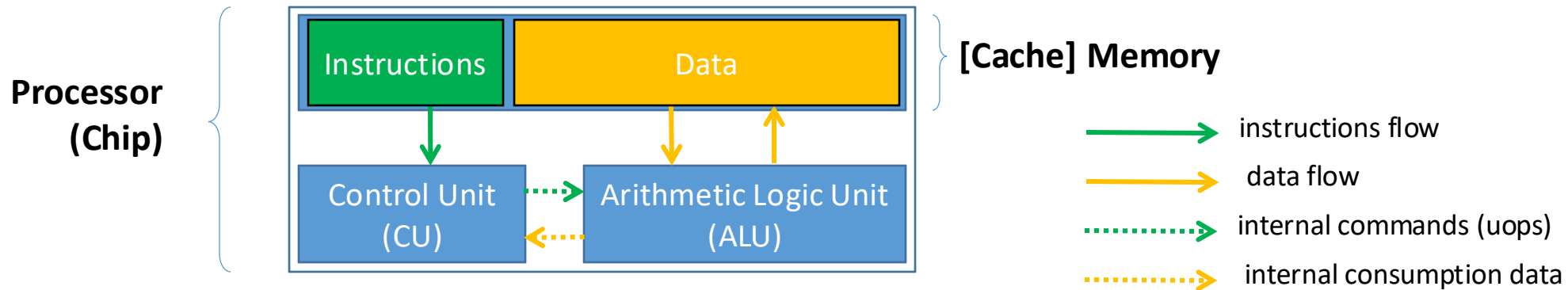
```
main:
    0, gp= 8      .frame      $fp,48,$31      # vars= 32, regs= 1/0, args=
                  .mask       0x40000000,-4
                  .fmask     0x00000000,0
                  .set        noreorder
                  .set        nomacro
                  addiu       $sp,$sp,-48
                  sw          $fp,44($sp)
                  move        $fp,$sp
                  lui          $2,%hi($LC0)
                  lwc1        $2,%lo($LC0)
...
                  mul.d       $f0,$f2,$f0
                  swc1        $f0,32($fp)
                  swc1        $f1,36($fp)
                  move        $2,$0
                  move        $sp,$fp
                  lw          $fp,44($sp)
                  addiu       $sp,$sp,48
                  j            $31
...
$LC0:
        .word      3649767765
        .word      1066523892
        .align     3
$LC1:
        .word      0
        .word      1078853632
        .align     3
$LC2:
        .word      0
        .word      1079902208
        .ident     "GCC: (Gentoo 4.6.3 p1.6, pie-0.5.2) 4.6.3"
```

Legend:
sw store word
addiu add immediate unsigned
mul.d multiply (double)

Computing requires **instructions** and **data**

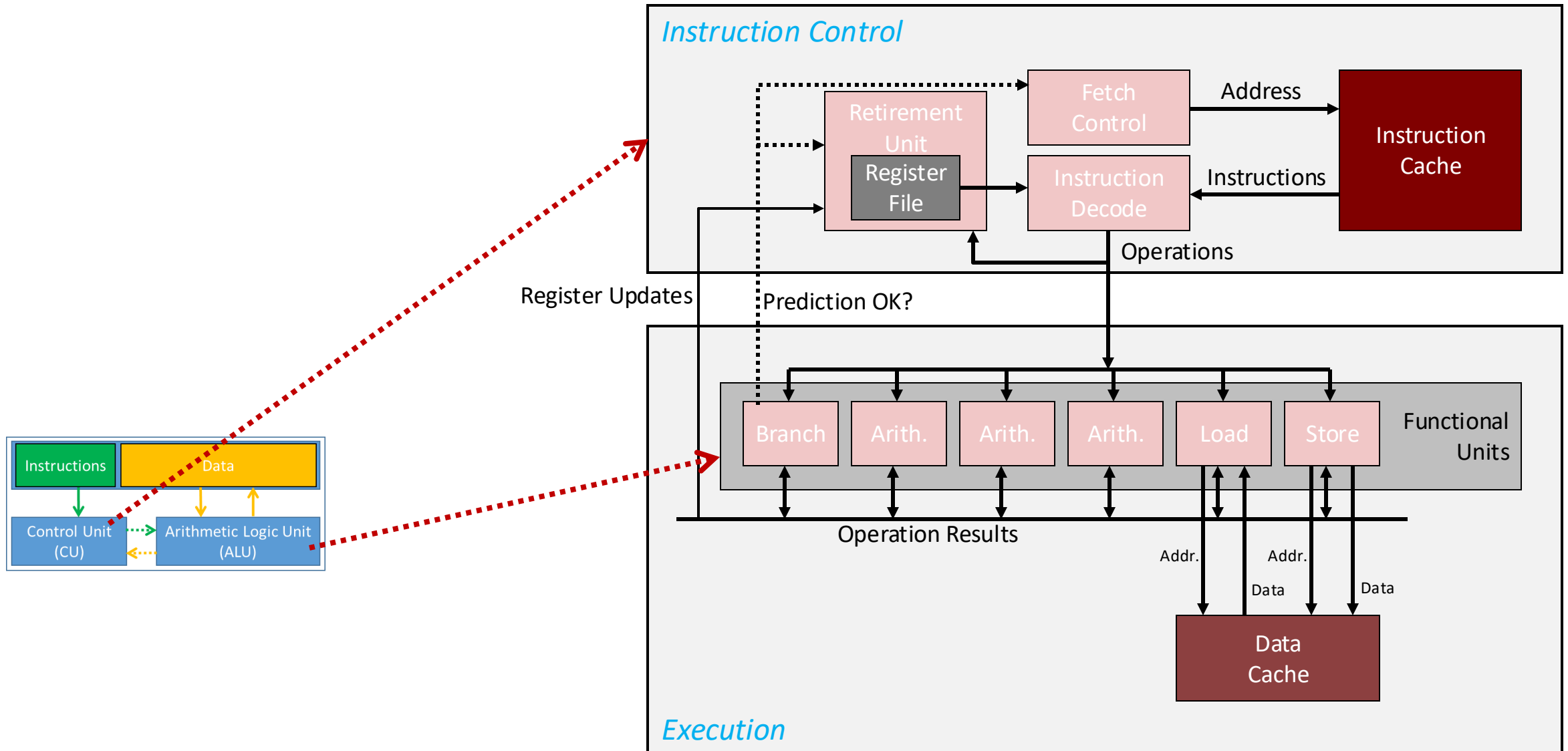
- An **instruction**: stored in memory as a binary string of 0 & 1 bits (32 or 64 bit-wide)
 - Instruction are fetched & decoded & executed
- A piece of **data**: stored in memory as string of 0 & 1 bits (8 bits, or 16, or 64, etc. – can be even user defined)
 - Data is used to produce results according to rules specified by the instructions

Computing Paradigm: Control Unit and Arithmetic Logic Unit



- Control unit (CU) controls “datapath” (i.e., hardware collection of functional units + registers + data buses)
 - Bring in the next instruction from memory
 - Decode the instruction (possibly breaking it up into uops)
 - Engage in actions to enable “instruction level parallelism” (ILP, more later)
- Arithmetic logic unit (ALU) is in charge of executing arithmetic and load/store operations
 - Made up of a bunch of “execution units” or “functional units”
 - EX: Arithmetic multiply/add unit (for integers), Arithmetic multiply/add unit (for floats), Special function unit called upon to compute $\sin$, $\cos$, $\exp$, $\log$, etc.

Schematic, middle-of-the-road CPU Microarchitecture



pop, sw, imul, lw, ... - What is this???

```
1 square(int):  
2     push    rbp  
3     mov     rbp, rsp  
4     mov     DWORD PTR [rbp-4], edi  
5     mov     eax, DWORD PTR [rbp-4]  
6     imul    eax, eax  
7     pop     rbp  
8     ret
```

- There is a limited set of instructions that the CU understands and thus acts upon
- This set of instructions makes up the **Instruction Set Architecture (ISA)**

Instruction Set Architecture (ISA)

- The same line of C code can lead to a different set of instructions on different processors (CPUs)
- This is so because two CPUs might implement two different ISAs
 - Intel's x86 architecture
 - MAC's M1/M2 arm architecture
 - Nvidia's PTX GPU architecture
- ISA: defines a “vocabulary” subsequently used by the compiler to specify the actions of a processor

ISA has Two important paradigms

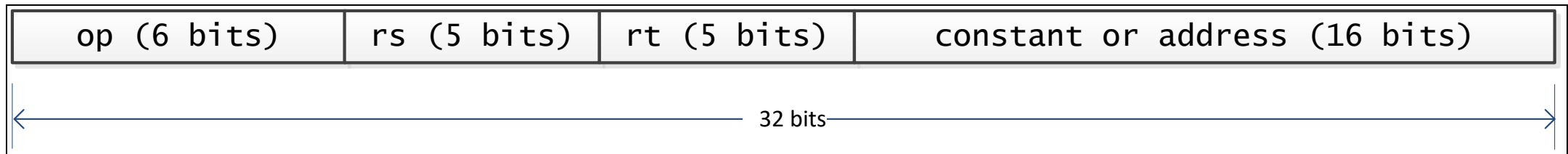
- **Reduced Instruction Set Computer (RISC):** a CPU architecture that uses a small, efficient set of simple instructions to improve performance
 - An instruction coded into a **fixed** set of bits (used to be 32, now it's 64)
 - Promoted by Stanford and UC Berkeley, in the form of the MIPS and SPARC architectures
 - EX: ARM Ltd: "Advanced RISC Machines"
- **Complex Instruction Set Computer (CISC):** a CPU architecture that uses a large, complex set of instructions, allowing each instruction to perform multiple operations or address multi-step tasks in a single command
 - Instructions have **various lengths**
 - Promoted by Intel and subsequently embraced & augmented by AMD
 - Ex: X86 is one successful CISC widely used in desktops
- RISC or CISC? Which one is better depends on your application targets

RISC vs. CISC

CISC	RISC
Emphasis on hardware	Emphasis on software
Multiple instruction sizes and formats	Instructions of same set with few formats
Fewer registers	Uses more registers
More addressing modes	Fewer addressing modes
Extensive use of microprogramming	Complexity in compiler
Instructions take a varying amount of cycle time	Instructions take one cycle time
Pipeline is more tricky, difficult	Pipeline is easy

OK, so what happens with an instruction?

- Each instruction goes through the **Fetch-Decode-Execute (FDX)** cycle
- FDX is what keeps the CU and ALU busy
 - FDX is done for instruction after instruction until program completes
- **Fetch**: an instruction is fetched from memory
 - Might look something like this (on 32 bits, MIPS, **lw \$t0, 12(\$s2)**):
100011100100100000000000000001100
- **Decode**: this string of 1s and 0s are decoded by the CU
 - Example: here's an "I" (eye) type instruction, made up of four fields

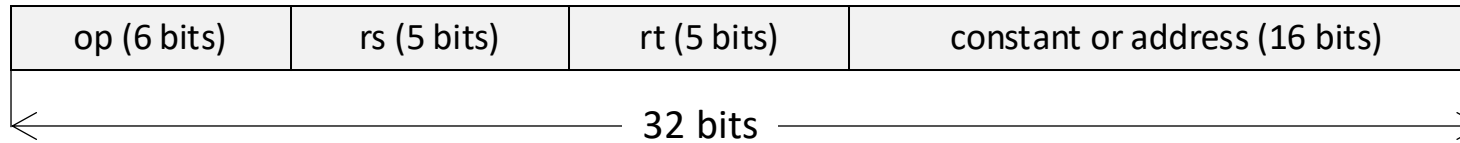


- **Execute**: once all data (operands) available, instruction is executed
 - The execution can take several cycles of the CPU [more later]

Instructions Types

- When an instruction is fetched and decoded, the process execute it based on its type
- Three main types of instructions in the MIPS ISA:
 - Type I (immediate): Instruction that involves immediate memory/value access
 - EX: load a value
 - Type R (register): Instruction that involves logical and arithmetic operations
 - EX: a += 6
 - Type J (jump): Instruction that involves jump/branch execution
 - EX: if(something is true) { ... }

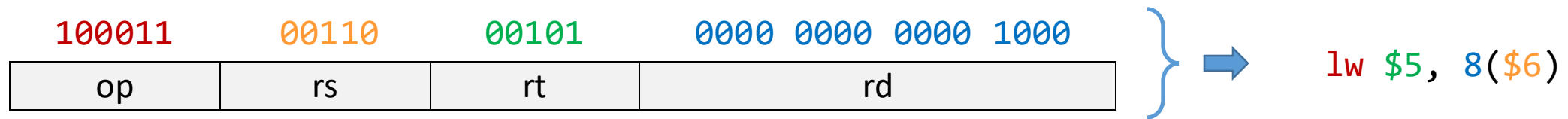
Type I (MIPS ISA)



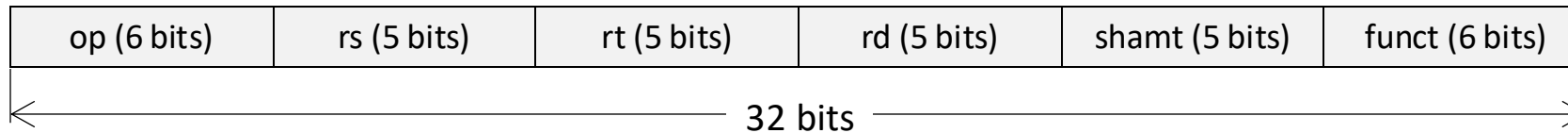
- The first six bits encode the basic operation to be completed (opcode)
- The next group of five bits - *rs*: source register to read data
- The subsequent group of five bits - *rt*: target register to store data
- The 16 bit “immediate” value, usually used as the offset value in various instructions
 - “Immediate” means that there is no need to read other registers or jump through other hoops. Immediately good to go
 - EX: accessing the value at `a[3]` will involve in an offset of $3*4=12$ bytes, assuming `a` is an integer array

Example, Type I (MIPS ISA)

- Example (look at the colors used at the left and right of the ➡ sign below; colors are matching left and right):
 - Load word from memory at the address \$6 + 8 into the register \$5

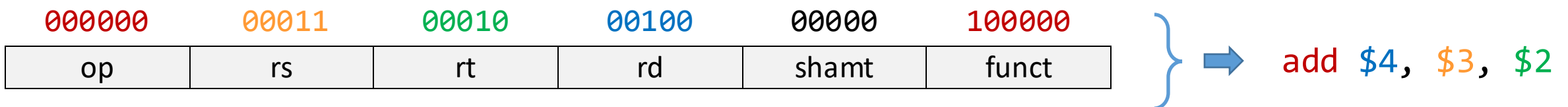


Type R (MIPS ISA)



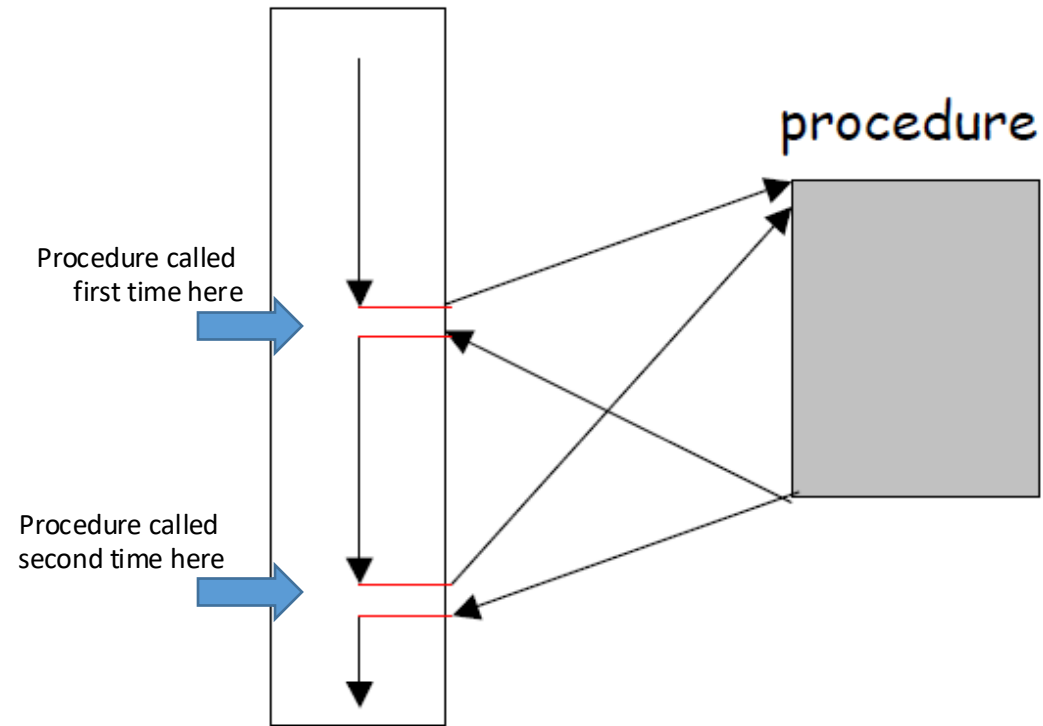
- What the fields encode:
 - **op** field encodes the operation code or opcode that selects a specific operation
 - **rs** and **rt** fields encode the first and second source registers; each field occupies five bits
 - Five bits for **rd** encode the destination register
 - Five bit shamt field (shift amount)
 - Six bit funct field, which is a function code that further qualifies the opcode

- Example:

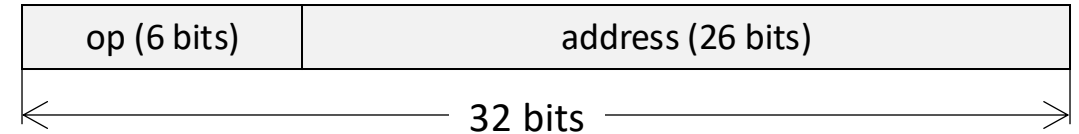


Type J (MIPS ISA)

Execution
instructions



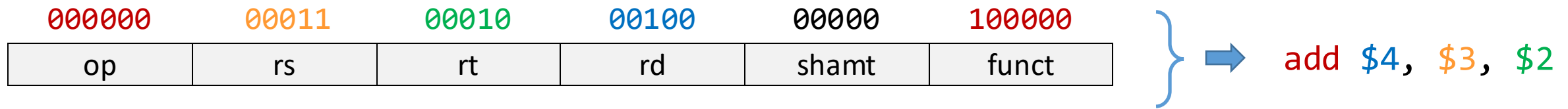
[<http://www.divms.uiowa.edu/~ghosh/1-24-06.pdf>]→



- Type J has an op field and an address field
- Jump instruction has a **word** address, **not an offset**
 - A 26-bit address field allows for jumps to any address between 0 and $2^{26}-1$
- Longer jumps use the “jump register” instruction:
`jr $ra # jump to address in register $ra`

How does CU Figure Out What Instruction Type to Deal With?

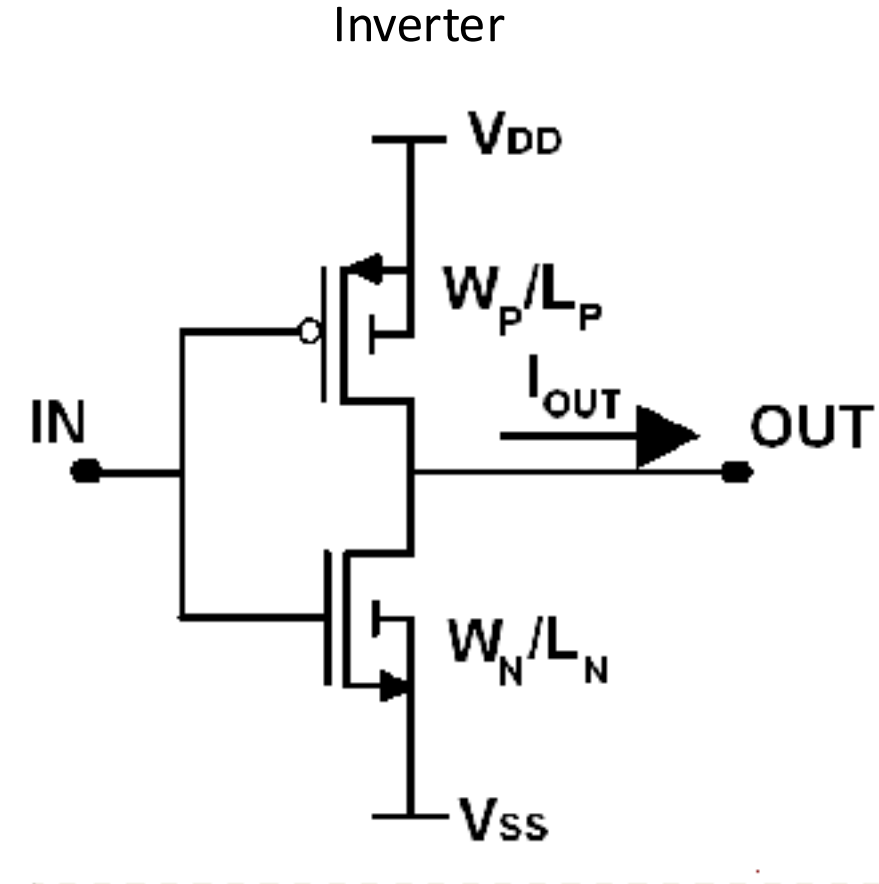
- The information is baked into the operation code (opcode), augmented by the funct field
 - Example



- ISA – defines a vocabulary
 - Specifies what a processor should be able to do
 - Load, store, jump on less than, etc.
- Microarchitecture – how the silicon is organized to implement the vocabulary embodied by the ISA
- Note: Intel and AMD both use the x86 ISA, but they work off different microarchitectures

How does FDX actually happen?

- Eventually, everything boils down to transistors
- Transistors are used to perform tasks (both CU and ALU)
 - A transistor is essentially a switch, like a light switch, controlled by an electrical current to perform a logic operation
 - With lots of transistors come opportunities for building and implementing in silicon additional functional/control units that can operate at the same time towards a shared goal



Transistors build up basic logic units: AND, OR, NOT

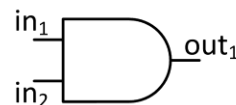
- Truth tables for bitwise AND, OR, and NOT

AND	in ₂ =0	in ₂ =1
in ₁ =0	0	0
in ₁ =1	0	1

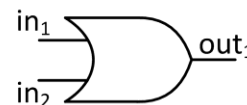
OR	in ₂ =0	in ₂ =1
in ₁ =0	0	1
in ₁ =1	1	1

NOT	
in ₁ =0	1
in ₁ =1	0

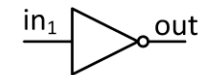
- The NOT logical operation is implemented using **two** transistors
 - One nMOS transistor:** Connects the output to the ground when the input is high
 - One pMOS transistor:** Connects the output to the power supply when the input is low
- AND and OR logical ops require more than **two** transistors



AND



OR



NOT

All Logic Functions can be Built on top of Fundamental Ones

- Easy to figure out the transistor setup once Logic Equation is available

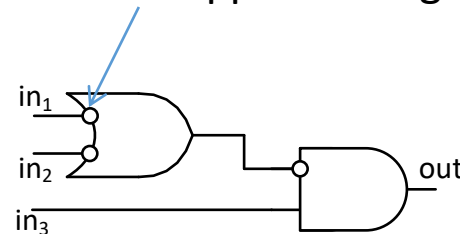
Truth Table

in ₁	in ₂	in ₃	Out
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

Logic Equation:

$$out = \overline{\overline{in_1} + \overline{in_2} \cdot in_3}$$

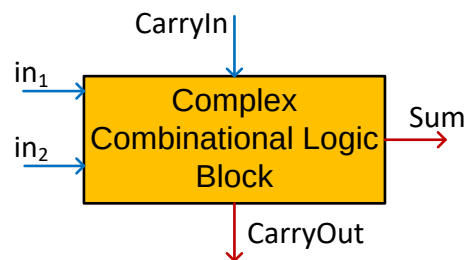
- Solution: digital logic block is a combination of AND, OR, and NOT gates
 - The NOT is represented as a circle **O** applied to signals moving down the bus



Another Example: One Bit Adder

visualize calculating:

```
  567+
  789
-----
 1356
```



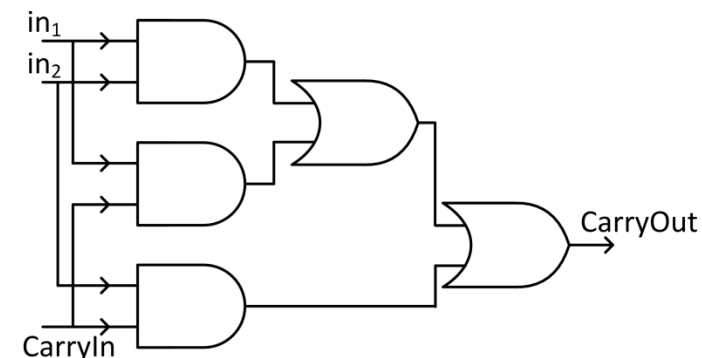
- Implement a digital circuit that produces the CarryOut digit in a one bit summation operation
 - Inputs: three of them (in_1 , in_2 , CarryIn)
 - Outputs: two of them (Sum, CarryOut)
 - Focus on CarryOut first

Truth Table

Inputs			Outputs		Comments
in_1	in_2	CarryIn	Sum	CarryOut	Sum is in base 2
0	0	0	0	0	
0	0	1	1	0	0+0 is 0; the CarryIn kicks in, makes the sum 1
0	1	0	1	0	
0	1	1	0	1	0+1 is 1, but CarryIn is 1; sum ends up being 0, CarryOut is 1.
1	0	0	1	0	
1	0	1	0	1	1+0 is 1, but CarryIn is 1; sum ends up being 0, CarryOut is 1.
1	1	0	0	1	1+1 is 0, carry 1.
1	1	1	1	1	1+1 is 0 and you CarryOut 1. Yet the CarryIn is 1, so the 0 in the sum becomes 1.

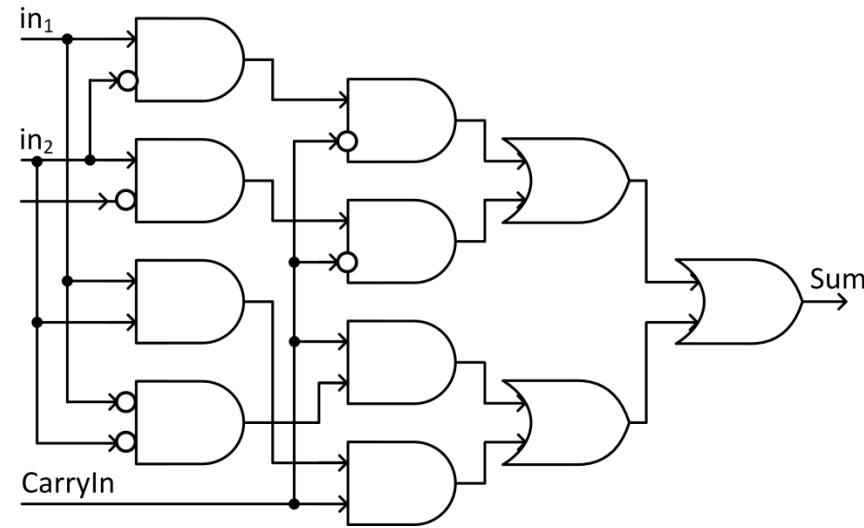
Logic Equation (for CarryOut):

$$\text{CarryOut} = (in_1 \cdot \text{CarryIn}) + (in_2 \cdot \text{CarryIn}) + (in_1 \cdot in_2)$$

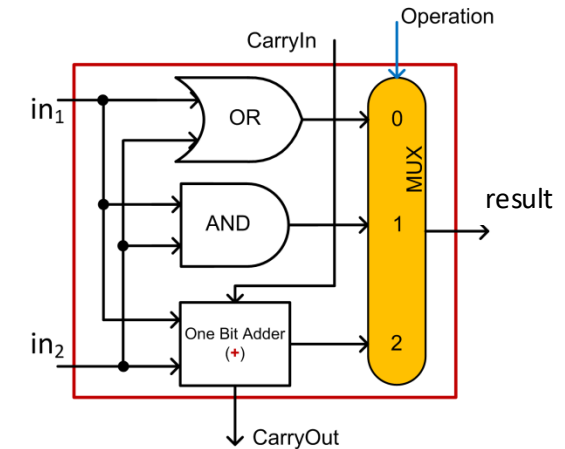


Another Example: One Bit Adder (cont'd)

- 1 Bit Adder, the Sum part

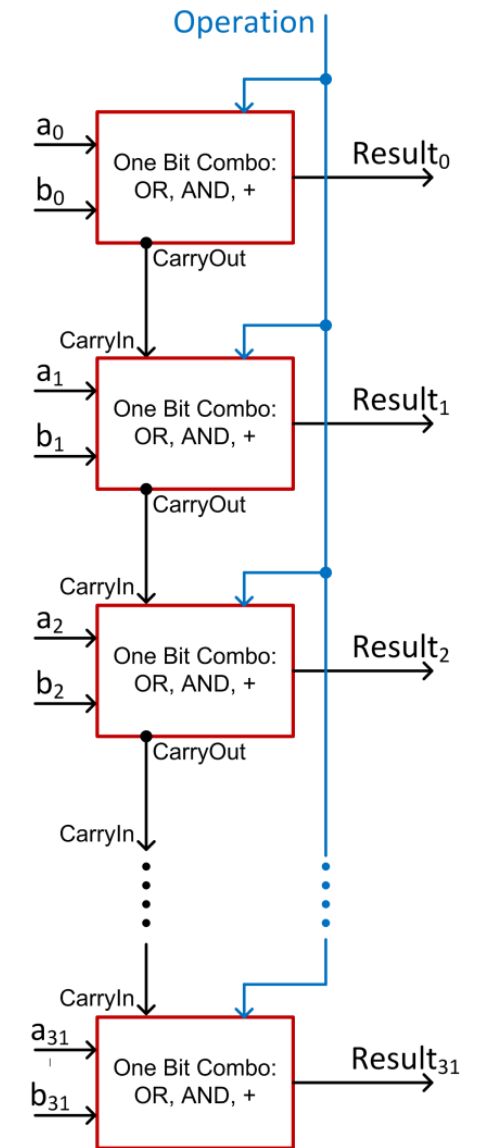
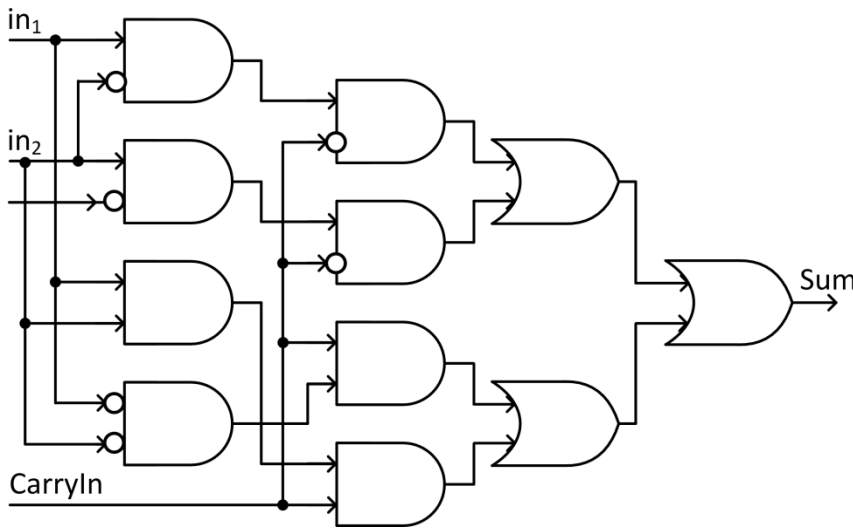
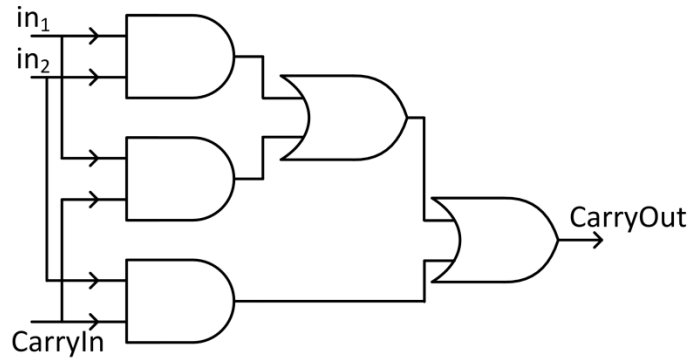


- A One Bit Combo: OR, AND, 1 Bit Sum
 - Controlled by the input "Operation"



Ripple Design of a 32-Bit Adder using Multiple 1-bit Adders

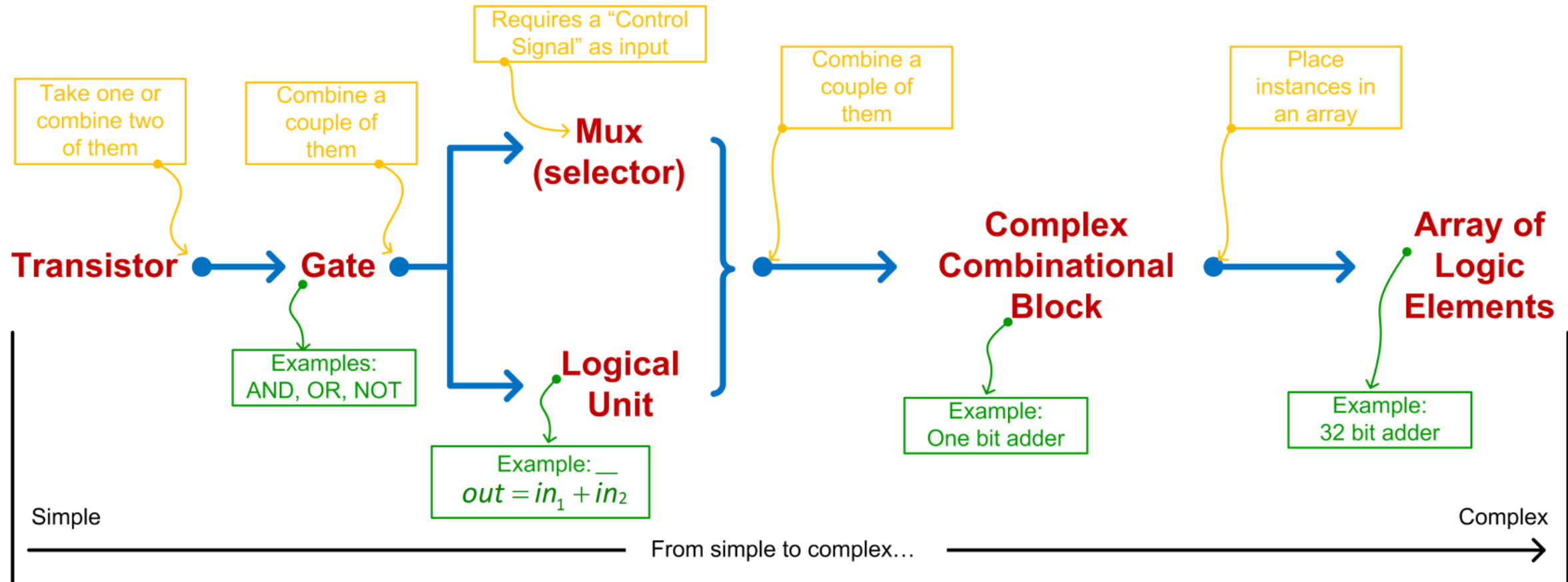
- Combine 32 of the 1-bit combos in an array of logic elements
 - Get one 32-bit unit that can do OR, AND, +
 - This “academic example” design requires about 1152 transistors



Observation of an Important Fact of running an Instruction

- Many instructions are not “atomic”
- You can’t just do the instruction in one shot – for example, in “add operation”
 - Recall the 32-bit adder chains 32 full adders in a cascade fashion
 - Completing an add operation can take several cycles through each individual 1-bit adder
 - Reason: The output of one bit adder influences the behavior of the downstream adder
- Something to keep in mind, motivates the idea of pipelining, yet to be discussed

Integrated Circuits: From Transistors to Chip Microarchitecture



In the end, it all boils down to transistors...

- For 50 years, every 18 months, number of transistors per unit area doubled (Moore's Law)
 - 2014-2015 technology: feature length was 14 nm (Intel)
 - 2018: 10 nm -12 nm
 - 2019-2020: 7 nm (top of the line - AMD)
 - 2021: 5 nm (see [interesting article](#))
 - 2024: 3 nm (TSMC)
 - Even farther ahead (18A?)
- Path forward unclear, what do we use when silicon-metal transistors can't get any smaller?
 - Maybe Carbon Nanotubes?
- Putting thing in perspective: back in 2017, I was telling students that 5 nm is the last frontier, and that's it...

Example, NVIDIA GPUs over time: Number of Transistors

- Fermi circ. 2010 – 2012 :
 - 40 nm technology
 - Up to 3 billion transistors → about 500 scalar processors, 0.5 d.p.Tflops
- Kepler circ. 2012 – 2014:
 - 28 nm technology
 - Chips w/ 7 billion transistors → about 2800 scalar processors, 1.5 d.p. Tflops
- Maxwell 2014 – 2016
 - 28 nm technology
 - Chips w/ 8 billion transistors → 3072 scalar processors, 6.1 s.p. Tflops
- Pascal 2016 – 2017
 - 16 nm technology
 - Chips w/ 12 billion transistors → 3584 scalar processors, 10.6 s.p. Tflops
- Turing 2018 – 2020
 - 12 nm technology
 - Chips w/ 19 billion transistors → 4608 scalar processors, 16 s.p. Tflops
- Ampere 2020 – 2023
 - 7 nm technology
 - Chips w/ 52 billion transistors → 6912 scalar processors, 19.5 s.p. Tflops
- Hopper 2023 –
 - 4 nm technology
 - Chips w/ 80 billion transistors → 16896 scalar processors, 60 s.p. Tflops

The Price of reaching 1 Gflop/second

- 1961: \$186 billion (adjusted for inflation, 2023 US dollars)
 - Combine 17 million IBM-1620 computers, at \$64K apiece (when adjusted for inflation)
- 2000: About \$1,000
 - Kentucky Linux Athlon Testbed, a 64+2 node Beowulf cluster built by the University of Kentucky COE in 2000
- 2015: 8 cents
 - Celeron G1830 & Radeon R9 295X2 System
- 2018: 6 cents
 - NVIDIA GTX 1080 Ti @ \$699 apiece
- 2023 (May): 1 cent
 - AMD Radeon RX 7600

GPU Acceleration: Ampere A100

[Obsolete as of 2023: Hopper is more recent – see <https://www.nvidia.com/en-us/data-center/h100/>]

- Ampere A100 GPU: 9,000,000,000,000 double precision FMA (fused multiply-add) operations per second
 - Note: $\text{FMA}(a, b, c) = (a \times b) + c$
- Very high bandwidth: 1555 GB/sec (HBM2)
 - Previous generation, Pascal had 720 GB/s
- Amazing at data processing (data parallelism)